DAY 4

Tuples and Dictionaries

4-page quick summary | for students new to Python

The big idea: **tuples are fixed records, dictionaries are labelled records.** Use a **tuple** when the data must not change and a **dict** when you want to look values up by name.

`tuple ( )`
ordered | cannot be changed | access by index

`dict { key: value }`
key-value pairs | changeable | access by key

1 Tuples (tuple): a fixed group of values

An ordered sequence that **cannot be changed** after creation. Duplicates are allowed. Written with parentheses `( )`. Good for fixed records and coordinates.

Create tuples correctly

```
one_item = ("Python",)
not_a_tuple = ("Python")
empty_tuple = ()
print(one_item, type(one_item))
print(not_a_tuple, type(not_a_tuple))
print(empty_tuple, type(empty_tuple))
```

```
OUTPUT
('Python',) <class 'tuple'>
Python <class 'str'>
() <class 'tuple'>
```

REMEMBER A one-item tuple **needs a comma**: `("Python",)`. Without it, `("Python")` is just a string in brackets.

Access items and slice

```
student = ("Priya", 19, "Data Science", 92, 87)
print(student[0], student[-1])
print(student[:2])
print(student[3:])
print(len(student), 100 in student)
```

```
OUTPUT
Priya 87
('Priya', 19)
(92, 87)
5 False
```

Indexing starts at **0**, **-1** is the last item, and a slice excludes its stop index (same rules as strings and lists).

Tuples are immutable

```
new = student[:2] + ("Maths",) + student[3:]
print(new)
```

```
OUTPUT
('Priya', 19, 'Maths', 92, 87)
```

WHY AN ERROR? `student[2] = 90` raises **TypeError: 'tuple' object does not support item assignment**. You cannot replace, append or remove items. If a change is needed, build a **new tuple** as above.

Tuple methods and built-in functions

Each example uses `marks = (85, 92, 78, 92, 88)`.

Tool	What it does	Example	Result
`.count(x)`	how many times x appears	`marks.count(92)`	`2`
`.index(x)`	index of the **first** x	`marks.index(92)`	`1`
`len()`	number of items	`len(marks)`	`5`
`min() / max()`	smallest / largest	`min(marks), max(marks)`	`78, 92`
`sum()`	total of numeric items	`sum(marks)`	`435`

Tuple unpacking

```
name, age, marks = ("Amit", 20, 78)
print(name, age, marks)

first, *middle, last = (10, 20, 30, 40, 50)
print(first, middle, last)
```

```
OUTPUT
Amit 20 78

10 [20, 30, 40] 50
```

KEY POINT The number of variables must match the number of items, unless one starts with `*`. The starred variable collects the leftover items into a **list**.

2 Dictionaries (dict): key-value pairs

Data stored as `{key: value}`. Keys are **unique** and **hashable** (strings, numbers, tuples of hashable values). Dictionaries are **mutable** and keep insertion order.

```
student = {"name": "Rahul", "age": 18, "marks": 85}
print(student)
print(student["name"], student["marks"])
```

```
OUTPUT
{'name': 'Rahul', 'age': 18, 'marks': 85}
Rahul 85
```

Access values: [ ] and get()

Each example uses `student = {"name": "Priya", "marks": 92}`.

Syntax	Example	Result
`[key]`	`student["marks"]`	`92`
`[key] (missing)`	`student["email"]`	`KeyError`
`.get(key)`	`student.get("email")`	`None`
`.get(key, default)`	`student.get("email", "Not provided")`	`'Not provided'`

Add and update entries

```
student = {"name": "Amit", "marks": 78}
student["course"] = "Python"
student["marks"] = 82
print(student)
student.update({"age": 20, "city": "Pune"})
print(student["age"], student["city"])
```

```
OUTPUT
{'name': 'Amit', 'marks': 82, 'course': 'Python'}
20 Pune
```

A **new key** adds an entry; an **existing key** updates its value. `update()` adds or changes several pairs at once. Keys are case-sensitive: `"Department"` and `"department"` are different keys.

Remove entries

Each example starts from `student = {"name": "Rahul", "age": 18, "marks": 85}`.

Method	What it does	Result
`.pop("marks")`	removes the key and returns its value	`returns 85`
`.popitem()`	removes and returns the last inserted pair	`returns ('marks', 85)`
`del student["age"]`	deletes a specific key (most preferred)	`{'name': 'Rahul', 'marks': 85}`
`.clear()`	removes all entries (use with caution)	`{}`

Nested dictionaries

```
students = {
    101: {"name": "Rahul", "marks": 85},
    102: {"name": "Priya", "marks": 92},
}
print(students[102])
print(students[102]["name"])
print(students[102]["marks"])
```

```
OUTPUT
{'name': 'Priya', 'marks': 92}
Priya
92
```

Useful dictionary methods

Method	Purpose	Method	Purpose
`.keys()`	view all keys	`.get(key, default)`	safely read a possibly missing key
`.values()`	view all values	`.update(other)`	add or update pairs
`.items()`	view key-value pairs	`.pop(key)`	remove a key and return its value
`.copy()`	create a shallow copy	`.clear()`	remove all entries

```
student = {"name": "Neha", "marks": 94}
print(student.keys())
print(student.values())
print(student.items())
print("name" in student, "email" in student)
```

```
OUTPUT
dict_keys(['name', 'marks'])
dict_values(['Neha', 94])
dict_items([('name', 'Neha'), ('marks', 94)])
True False
```

Loop through a dictionary

```
student = {"name": "Neha", "marks": 94}
for key, value in student.items():
    print(key, "->", value)
for key in student:
    print(key)
```

```
OUTPUT
name -> Neha
marks -> 94
name
marks
```

`for key in d` loops over keys, `d.values()` over values, `d.items()` over key-value pairs.

3 Tuple vs dictionary

Feature	Tuple	Dictionary
Stores	ordered values	key-value pairs
Access	by index: record[0]	by key: student["name"]
Mutable?	No	Yes
Duplicates	allowed	values may repeat; **keys must be unique**
Good for	fixed records, coordinates	named fields, lookup tables

4 Mini project: student report

A **dict** for named fields, a **tuple** to keep the subjects fixed, and a **nested dict** for marks.

```
student = {
    "id": 101,
    "name": "Rahul Sharma",
    "subjects": ("Python", "SQL", "Excel"),
    "marks": {"Python": 85, "SQL": 90, "Excel": 88}
}
total = sum(student["marks"].values())
average = total / len(student["marks"])
print(f"Student: {student['name']}")
print(f"Student ID: {student['id']}")
print("Marks:", student["marks"])
print(f"Total marks: {total}")
print(f"Average marks: {average:.2f}")
```

```
OUTPUT
Student: Rahul Sharma
Student ID: 101
Marks: {'Python': 85, 'SQL': 90, 'Excel': 88}
Total marks: 263
Average marks: 87.67
```

5 Mistakes, practice and recap

Spot the bug

Buggy line	Problem	Fix
`x = ("Python")`	this is a str, not a tuple	`x = ("Python",)`
`marks[0] = 90 (tuple)`	TypeError: tuples cannot be changed	`build a new tuple`
`student["email"] (missing)`	KeyError: the key does not exist	`student.get("email")`
`a, b = (1, 2, 3)`	ValueError: too many values to unpack	`a, b, c = (1, 2, 3)`
`student["Name"]`	KeyError: "Name" is not "name"	`student["name"]`

Practice questions: model solutions

Your code may differ and still be right. Compare the **idea**, not the wording.

Tuple 1-2: five days; first and last

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri")
print(days[0], days[-1])
```

Tuple 3-4: one-item tuple; count 90

```
one = ("Python",)
print((90, 75, 90, 88, 90).count(90))
```

Tuple 5-6: unpack; why the error?

```
name, age, marks = ("Neha", 17, 94)
# my_tuple[0] = "New value" fails:
# tuples are immutable
```

Dict 1-2: create; print the name

```
student = {"name": "Priya", "age": 21,
           "course": "Python", "marks": 85}
print(student["name"])
```

Dict 3-4: get default; add grade, update marks

```
print(student.get("email", "Not provided"))
student["grade"] = "A"
student["marks"] = 90
```

Dict 5: loop through keys and values

```
for key, value in student.items():
    print(key, value)
```

Dict 6: nested dict with IDs as keys

```
records = {1: {"name": "Rahul", "marks": 85},
           2: {"name": "Priya", "marks": 92}}
```

Class Problems 1-6: topics

1 create and index a tuple | 2 tuple slicing | 3 tuple unpacking | 4 create and access a dict | 5 add and update values | 6 keys(), values(), pop()

Recap quiz: answers

#	Question	Answer
1	Which syntax creates a one-item tuple?	`("Python",)`
2	Can a tuple item be reassigned?	`No`
3	What does student["email"] do if the key is missing?	`raises KeyError`
4	Which method safely reads a possibly missing key?	`.get(key, default)`
5	What does .items() provide?	`key-value pairs`
6	How do you add or update several entries at once?	`.update({...})`

Learning checklist: tick each box when you can do it confidently

- [ ] Create, access, slice and unpack tuples.
- [ ] Explain tuple immutability and use count() and index().
- [ ] Create dictionaries and access values using keys.
- [ ] Add, update, remove and loop through dictionary entries.
- [ ] Use safe lookups, nested structures and common dictionary methods.